DATA SCIENCE RESOURCES

Pandas Interview Questions Guide

Are you confused about whatto study before your next data analytics interview?

This guide breaks down exactly what you should revise in Pandas through beginner-friendly interview questions covering data manipulation, filtering, grouping, missing data handling, merging, and core concepts.
Use this as a revision checklist, not something to memorize line by line.

If you can explain these topics clearly, you're covering what most interviews actually test.

Ideal For:

- Data Analysts
- Data Scientists
- Business Analysts
- Python/Data Engineering roles

---

1. Pandas Basics (Foundations)

Q1. What is Pandas and why is it used in data analysis?

Answer: Pandas is a Python library for data manipulation and analysis. It provides two primary data structures - DataFrame and Series - that make working with structured data intuitive and efficient.

Why it's used:

- Handles tabular data similar to Excel or SQL tables
- Powerful data cleaning and transformation capabilities
- Efficient handling of missing data
- Easy data import/export (CSV, Excel, SQL, JSON)
- Built on NumPy for fast numerical operations
- Integrates well with visualization libraries

Learning Resources:

- Official Pandas Documentation: https://pandas.pydata.org/docs/
- Pandas Getting Started Tutorial: https://pandas.pydata.org/docs/getting_started/intro_tutorials/

Q2. What is a DataFrame? How is it different from a Series?

Answer:

DataFrame:

- Two-dimensional labeled data structure
- Contains rows and columns (like a spreadsheet or SQL table)
- Each column can have different data types
- Think of it as a collection of Series

Series:

- One-dimensional labeled array
- Contains a single column of data
- Has a consistent data type
- Think of it as a single column from a DataFrame

Key Differences:

- DataFrame: 2D (rows × columns), Series: 1D (single column)
- DataFrame can hold multiple data types, Series has one type
- DataFrame has both row and column labels, Series has only index labels

Example:

```
# Series - single column
s = pd.Series([10, 20, 30], name='Age')

# DataFrame - multiple columns
df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'City': ['NYC', 'LA', 'Chicago']
})
```

Q3. How do you create a DataFrame from a dictionary or list?

Answer: From Dictionary:

```
import pandas as pd

# Dictionary with lists as values
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'Salary': [50000, 60000, 70000]
}         df         =
pd.DataFrame(data)
```

From List of Dictionaries:

```
data = [
    {'Name': 'Alice', 'Age': 25},
    {'Name': 'Bob', 'Age': 30},
    {'Name': 'Charlie', 'Age': 35}
]         df         =
pd.DataFrame(data)
```

From List of Lists:

```
data = [
    ['Alice', 25, 50000],
    ['Bob', 30, 60000],
    ['Charlie', 35, 70000]
]   df   =   pd.DataFrame(data,   columns=['Name',   'Age',
'Salary'])
```

Q4. How do you read a CSV file using Pandas?

Answer:

Use the `pd.read_csv()` function:

```
import pandas as pd

# Basic usage
df = pd.read_csv('file.csv')

# Common parameters
df = pd.read_csv('file.csv',
                 sep=',',              # Delimiter (default is comma)
                 header=0,             # Row number to use as column names
                 index_col=0,          # Column to use as row index
                 usecols=['A', 'B'],   # Columns to read
                 na_values=['NA', ''], # Additional strings to recognize as NaN
                 skiprows=2,           # Rows to skip at start
                 nrows=1000)           # Number of rows to read
```

Learning Resources:

- Pandas IO Tools Documentation: https://pandas.pydata.org/docs/user_guide/io.html
- Real Python Pandas Read CSV Tutorial: https://realpython.com/pandas-read-write-files/

Q5. How do you check the number of rows and columns in a DataFrame?

Answer:

Use the `.shape` attribute:

```
df.shape
# Returns: (rows, columns) as a tuple
# Example output: (100, 5) means 100 rows and 5 columns

# Get just rows
num_rows = df.shape[0]

# Get just columns
num_columns = df.shape[1]

# Alternative for number of rows
len(df)
```

Q6. How do you display the first and last few rows of data?

Answer: First

rows - `head()` :

```
df.head()      # First 5 rows (default)
df.head(10)    # First 10 rows
```

Last rows - `tail()` :

```
df.tail()      # Last 5 rows (default)
df.tail(10)    # Last 10 rows
```

Why use them:

- Quick data preview
- Verify data loaded correctly
- Check structure and format
- Useful during exploratory data analysis

Q7. How do you check column names in a DataFrame?

Answer:

```
# Get column names
df.columns
# Returns: Index object with column names

# Convert to list
column_list = df.columns.tolist()

# Number of columns
len(df.columns)

# Check if column exists
'Age' in df.columns  # Returns True or False
```

Q8. How do you check data types of all columns?

Answer:

```
# Get data types of all columns
df.dtypes

# Common data types in Pandas:
# - int64: Integer numbers
# - float64: Decimal numbers
# - object: Strings or mixed types
# - bool: Boolean (True/False)
# - datetime64: Date and time
# - category: Categorical data

# Check specific column type
df['Age'].dtype

# Get summary with data types
df.info()  # Also shows non-null counts
```

Q9. How do you get a quick summary of the dataset?

Answer:

Use `describe()` for numerical columns:

```
df.describe()
# Returns statistics for numeric columns:
# - count: Number of non-null values
# - mean: Average value
# - std: Standard deviation
# - min: Minimum value
# - 25%, 50%, 75%: Quartiles
# - max: Maximum value

# Include all columns (including non-numeric)
df.describe(include='all')

# Only specific columns
df[['Age', 'Salary']].describe()
```

Learning Resources:

- Pandas Descriptive Statistics: https://pandas.pydata.org/docs/user_guide/basics.html#descriptive-statistics

---

Q10. What does `info()` tell you about the data?

Answer:

`info()` provides a concise summary of the DataFrame:

```
df.info()
```

Information provided:

- Number of rows and columns
- Column names
- Data type of each column
- Number of non-null values (helps identify missing data)
- Memory usage

Why it's useful:

- Quick overview of dataset structure
- Identify columns with missing values
- Check data types before analysis
- Estimate memory requirements

Q11. How do you rename columns in Pandas?

Answer:

Using `rename()` method:

```
# Rename specific columns
df.rename(columns={'old_name': 'new_name', 'age': 'Age'}, inplace=True)

# Without inplace (returns new DataFrame)
df_new = df.rename(columns={'old_name': 'new_name'})
```

Rename all columns at once:

```
df.columns = ['Name', 'Age', 'City', 'Salary']
```

Make all column names lowercase:

```
df.columns = df.columns.str.lower()
```

Replace spaces with underscores:

```
df.columns = df.columns.str.replace(' ', '_')
```

Q12. How do you change the index of a DataFrame?

Answer: Set a column as

index:

```
df.set_index('column_name', inplace=True)
```

Reset index (convert index back to column):

```
df.reset_index(inplace=True)
```

Set custom index:

```
df.index = ['A', 'B', 'C', 'D']
```

Set range as index:

```
df.index = range(1, len(df) + 1)
```

---

2. Indexing & Data Selection

Q1. How do you select a single column from a DataFrame?

Answer: Two ways to select a single column:

```
# Method 1: Using bracket notation (returns Series)
df['column_name']

# Method 2: Using dot notation (returns Series)
df.column_name

# Get as DataFrame (single column)
df[['column_name']]
```

Q2. How do you select multiple columns at once?

Answer: Pass a list of column names:

```
# Select multiple columns
df[['Name', 'Age', 'Salary']]

# Store column list in variable
cols = ['Name', 'Age']
df[cols]

# Select columns by position
df.iloc[:, [0, 2, 4]]  # Columns at index 0, 2, 4
```

Q3. Difference between `loc` and `iloc`?

Answer:

`loc` - Label-based indexing:

- Uses row/column labels (names)
- Inclusive of end point
- Example: `df.loc[0:5, 'Name']` includes row 5

`iloc` - Integer position-based indexing:

- Uses integer positions (0-indexed)
- Exclusive of end point
- Example: `df.iloc[0:5, 0]` excludes row 5

Examples:

```
# loc - by label
df.loc[0, 'Name']           # Single value
df.loc[0:5, ['Name', 'Age']] # Rows 0-5, specific columns

# iloc - by position
df.iloc[0, 0]               # First row, first column
df.iloc[0:5, 0:2]           # First 5 rows, first 2 columns
```

Learning Resources:

- Pandas Indexing Documentation: https://pandas.pydata.org/docs/user_guide/indexing.html

---

Q4. How do you filter rows based on a condition?

Answer: Use boolean

indexing:

```
# Single condition
df[df['Age'] > 25]

# Store condition in variable
condition = df['Age'] > 25
filtered_df = df[condition]

# Common conditions
df[df['City'] == 'NYC']          # Equal to
df[df['Salary'] >= 50000]        # Greater than or equal
df[df['Name'].str.startswith('A')] # String condition
```

Q5. How do you filter rows using multiple conditions?

Answer: Use `&` (AND), `|` (OR), and `~` (NOT) operators:

```
# AND condition (both must be true)
df[(df['Age'] > 25) & (df['City'] == 'NYC')]

# OR condition (either can be true)
df[(df['Age'] > 25) | (df['Salary'] > 60000)]

# NOT condition (negate)
df[~(df['City'] == 'NYC')]  # All except NYC

# Multiple AND conditions
df[(df['Age'] > 25) & (df['City'] == 'NYC') & (df['Salary'] > 50000)]
```

Important: Use `&` and `|`, not `and` and `or`. Always use parentheses around each condition.

Q6. How do you select rows by index position?

Answer:

Use `iloc`:

```
# Single row by position
df.iloc[0]        # First row
df.iloc[-1]       # Last row

# Multiple rows by position
df.iloc[0:5]      # First 5 rows (0-4)
df.iloc[[0, 2, 4]] # Specific rows

# All rows, specific columns
df.iloc[:, 0:3]   # All rows, first 3 columns
```

Q7. How do you select rows by label?

Answer:

Use `loc`:

```
# Single row by label
df.loc[0]         # Row with index label 0

# Multiple rows by label
df.loc[0:5]       # Rows 0 through 5 (inclusive)
df.loc[[0, 2, 4]] # Specific rows

# Rows and columns by label
df.loc[0:5, ['Name', 'Age']]
```

Q8. How do you sort a DataFrame by one column?

Answer:

Use `sort_values()`:

```
# Sort by single column (ascending)
df.sort_values('Age')

# Sort descending
df.sort_values('Age', ascending=False)

# Sort and save changes
df.sort_values('Age', inplace=True)

# Sort by index
df.sort_index()
```

Q9. How do you sort by multiple columns?

Answer: Pass a list of column names:

```
# Sort by multiple columns
df.sort_values(['City', 'Age'])

# Different sort orders for each column
df.sort_values(['City', 'Age'], ascending=[True, False])

# City ascending, Age descending
```

Q10. How do you select rows where a value is in a list?

Answer:

Use the `isin()` method:

```
# Check if values are in a list
cities = ['NYC', 'LA', 'Chicago']
df[df['City'].isin(cities)]

# Select rows NOT in list
df[~df['City'].isin(cities)]

# Multiple columns
df[df['City'].isin(['NYC', 'LA']) & df['Age'].isin([25, 30])]
```

Q11. How do you select rows between two values?

Answer:

Use `between()` method or comparison operators:

```
# Using between (inclusive)
df[df['Age'].between(25, 35)]

# Using comparison operators
df[(df['Age'] >= 25) & (df['Age'] <= 35)]

# Date ranges
df[df['Date'].between('2024-01-01', '2024-12-31')]
```

Q12. What happens if the index is not unique?

Answer: If the index is not

unique: Problems:

- `loc` returns multiple rows instead of single row
- Can lead to unexpected results
- Makes data selection ambiguous
- Affects merge/join operations

Example:

```
# If multiple rows have index 0
df.loc[0]  # Returns DataFrame with all rows where index=0

# Check for duplicate indexes
df.index.is_unique  # Returns False if duplicates exist
df.index.duplicated().sum()  # Count duplicate indexes

# Fix: Reset index
df.reset_index(drop=True, inplace=True)
```

Best Practice: Keep indexes unique or use `reset_index()` when needed.

---

3. Handling Missing Data

Q1. How do you check for missing values in Pandas?

Answer:

Use `isnull()` or `isna()` :

```
# Check for missing values (returns True/False for each cell)
df.isnull()
df.isna()  # Same as isnull()

# Count missing values per column
df.isnull().sum()

# Total missing values in entire DataFrame
df.isnull().sum().sum()

# Check if any missing values exist
df.isnull().any()

# Visualize missing data
df.isnull().sum().plot(kind='bar')
```

Learning Resources:

- Pandas Missing Data Tutorial: https://pandas.pydata.org/docs/user_guide/missing_data.html

---

Q2. What is the difference between `isnull()` and `isna()`?

Answer: No difference - they are

aliases:

- isnull() and isna() do exactly the same thing
- Both check for missing values (NaN, None, NaT)
- Use whichever you prefer for consistency

```
# Both produce identical results
df.isnull()
df.isna()

# Same for inverse
df.notnull()
df.notna()
```

Recommendation: Pick one and use it consistently in your code.

Q3. How do you count missing values in each column?

Answer:

Combine `isnull()` with `sum()` :

```
# Count missing values per column
df.isnull().sum()

# As percentage
(df.isnull().sum() / len(df)) * 100

# Show only columns with missing values
df.isnull().sum()[df.isnull().sum() > 0]

# Sort by number of missing values
df.isnull().sum().sort_values(ascending=False)
```

Q4. How do you drop rows with missing values?

Answer:

Use `dropna()` :

```
# Drop rows with ANY missing values
df.dropna()

# Drop rows where ALL values are missing
df.dropna(how='all')

# Drop rows with missing values in specific columns
df.dropna(subset=['Age', 'Salary'])

# Require at least N non-null values
df.dropna(thresh=3)  # Keep rows with at least 3 non-null values

# Save changes
df.dropna(inplace=True)
```

Q5. How do you drop columns with missing values?

Answer:

Use `dropna()` with `axis=1` :

```
# Drop columns with ANY missing values
df.dropna(axis=1)

# Drop columns where ALL values are missing
df.dropna(axis=1, how='all')

# Drop columns with more than 50% missing
threshold = len(df) * 0.5
df.dropna(axis=1, thresh=threshold)
```

Q6. How do you fill missing values with a constant?

Answer:

Use `fillna()` :

```
# Fill all missing values with 0
df.fillna(0)

# Fill specific column
df['Age'].fillna(0)

# Fill different columns with different values
df.fillna({'Age': 0, 'Salary': 50000, 'City': 'Unknown'})

# Save changes
df.fillna(0, inplace=True)
```

Q7. How do you fill missing values with mean or median?

Answer: Calculate statistic and use `fillna()` :

```
# Fill with mean
df['Age'].fillna(df['Age'].mean(), inplace=True)

# Fill with median
df['Salary'].fillna(df['Salary'].median(), inplace=True)

# Fill with mode (most frequent value)
df['City'].fillna(df['City'].mode()[0], inplace=True)

# Fill multiple columns
df.fillna({
    'Age': df['Age'].mean(),
    'Salary': df['Salary'].median()
})
```

Q8. How do you forward fill missing values?

Answer:

Use `fillna(method='ffill')` or `ffill()`:

```
# Forward fill (use previous value)
df.fillna(method='ffill')
df.ffill()  # Shorthand

# Forward fill specific column
df['Age'].ffill()

# Limit number of consecutive fills
df.ffill(limit=2)  # Fill maximum 2 consecutive NaNs
```

When to use: Time series data where you want to carry forward the last known value.

Q9. How do you backward fill missing values?

Answer:

Use `fillna(method='bfill')` or `bfill()`:

```
# Backward fill (use next value)
df.fillna(method='bfill')
df.bfill()  # Shorthand

# Backward fill specific column
df['Age'].bfill()

# Limit number of consecutive fills
df.bfill(limit=2)
```

When to use: When future values are more relevant than past values.

Q10. When should you drop missing values instead of filling them?

Answer: Drop

when:

- Very few missing values (< 5% of data)
- Missing data is completely random
- Filling would introduce bias
- Specific analysis requires complete data
- Missing values represent data collection errors

Fill when:

- Large proportion of missing data (dropping loses too much information)
- Missing data follows a pattern
- Domain knowledge suggests appropriate fill value
- Time series where continuity matters

Consider:

- Impact on analysis
- Sample size after dropping
- Reason for missingness
- Downstream model requirements

Q11. How does Pandas treat missing values during calculations?

Answer: Pandas handles missing values automatically:

Default behavior:

- Arithmetic operations skip NaN values
- `sum()`, `mean()`, etc. ignore NaN
- Comparisons with NaN return False
- NaN propagates in some operations

Examples:

```
# Sum ignores NaN
df['Age'].sum()  # Adds only non-null values

# Mean automatically excludes NaN
df['Salary'].mean()

# Count excludes NaN
df['Age'].count()  # Only non-null values

# To include NaN in count
len(df['Age'])
```

Override behavior:

```
# Include NaN in sum (treats as 0)
df['Age'].sum(skipna=False)  # Returns NaN if any NaN exists
```

Q12. How do missing values affect groupby operations?

Answer: By default, `groupby()` excludes missing values:

```
# NaN values in groupby column are excluded
df.groupby('City')['Salary'].mean()
# Rows where City is NaN are not included

# Include NaN as a group
df.groupby('City', dropna=False)['Salary'].mean()

# Missing values in aggregated columns are skipped
df.groupby('City')['Salary'].sum()
# NaN values in Salary are ignored in sum
```

Important: Understand how missing values affect your grouping logic and aggregations.

---

4. Data Cleaning & Preparation

Q1. How do you remove duplicate rows from a DataFrame?

Answer:

Use `drop_duplicates()`:

```
# Remove duplicate rows (keeps first occurrence)
df.drop_duplicates()

# Keep last occurrence
df.drop_duplicates(keep='last')

# Remove all duplicates (don't keep any)
df.drop_duplicates(keep=False)

# Save changes
df.drop_duplicates(inplace=True)

# Check for duplicates first
df.duplicated().sum()  # Count duplicate rows
```

Q2. How do you remove duplicates based on specific columns?

Answer: Pass column names to `drop_duplicates()` :

```
# Remove duplicates based on specific column
df.drop_duplicates(subset=['Email'])

# Based on multiple columns
df.drop_duplicates(subset=['Name', 'Email'])

# Keep last occurrence
df.drop_duplicates(subset=['Email'], keep='last')

# Find duplicates in specific column
df[df.duplicated(subset=['Email'])]
```

Q3. How do you change a column's data type?

Answer:

Use `astype()` :

```
# Convert to integer
df['Age'] = df['Age'].astype(int)

# Convert to float
df['Salary'] = df['Salary'].astype(float)

# Convert to string
df['ID'] = df['ID'].astype(str)

# Convert multiple columns
df = df.astype({'Age': int, 'Salary': float, 'City': str})

# Handle errors
df['Age'] = pd.to_numeric(df['Age'], errors='coerce')  # Invalid → NaN
```

Learning Resources:

- Pandas Data Types Tutorial: https://pandas.pydata.org/docs/user_guide/basics.html#dtypes

---

Q4. How do you convert a column to datetime format?

Answer:

Use `pd.to_datetime()` :

```
# Convert to datetime
df['Date'] = pd.to_datetime(df['Date'])

# Specify format for faster parsing
df['Date'] = pd.to_datetime(df['Date'], format='%Y-%m-%d')

# Handle errors
df['Date'] = pd.to_datetime(df['Date'], errors='coerce')  # Invalid → NaT

# Common format codes:
# %Y: Year (4 digits)
# %m: Month (01-12) #
%d: Day (01-31) # %H:
Hour  (00-23)  #  %M:
Minute  (00-59)  #  %S:
Second (00-59)
```

Q5. How do you clean string columns (lowercase, strip spaces)?

Answer: Use string methods with `.str` accessor:

```
# Convert to lowercase
df['Name'] = df['Name'].str.lower()

# Convert to uppercase
df['Name'] = df['Name'].str.upper()

# Remove leading/trailing spaces
df['Name'] = df['Name'].str.strip()

# Remove all spaces
df['Name'] = df['Name'].str.replace(' ', '')

# Chain multiple operations
df['Email'] = df['Email'].str.lower().str.strip()
```

Q6. How do you replace values in a column?

Answer:

Use `replace()`:

```
# Replace single value
df['City'] = df['City'].replace('NYC', 'New York')

# Replace multiple values
df['City'] = df['City'].replace({
    'NYC': 'New York',
    'LA': 'Los Angeles',
    'SF': 'San Francisco'
})

# Replace in entire DataFrame
df.replace('Unknown', np.nan, inplace=True)

# Replace using regex
df['Phone'] = df['Phone'].str.replace(r'\D', '', regex=True)  # Remove non-digits
```

Q7. How do you handle inconsistent categorical values?

Answer: Standardize values before analysis:

```
 # Standardize case
df['Gender'] = df['Gender'].str.lower().str.strip()

# Map variations to standard values
gender_mapping = {
    'm': 'Male',
    'male': 'Male',
    'f': 'Female',
    'female': 'Female'
}                         df['Gender']                         =
df['Gender'].str.lower().replace(gender_mapping)

# Use categories
df['Status'] = df['Status'].astype('category')

#   Check   unique   values
df['Gender'].unique()
df['Gender'].value_counts()
```

Q8. How do you remove special characters from strings?

Answer: Use string methods with regex:

```
 # Remove specific characters
df['Text'] = df['Text'].str.replace('[#@$%]', '', regex=True)

# Keep only alphanumeric
df['Text'] = df['Text'].str.replace('[^A-Za-z0-9 ]', '', regex=True)

# Remove punctuation
df['Text'] = df['Text'].str.replace('[^\w\s]', '', regex=True)

# Remove digits
df['Text'] = df['Text'].str.replace('\d+', '', regex=True)
```

Q9. How do you split a column into multiple columns?

Answer:

Use `str.split()` :

```
 # Split by delimiter
df[['First_Name', 'Last_Name']] = df['Name'].str.split(' ', expand=True)

# Split by comma
df[['City', 'State']] = df['Location'].str.split(',', expand=True)

# Limit splits
df[['Area_Code', 'Number']] = df['Phone'].str.split('-', n=1, expand=True)

# Split and keep first/last part df['First_Name'] =
df['Name'].str.split('  ').str[0]  df['Last_Name']  =
df['Name'].str.split(' ').str[-1]
```

Q10. How do you merge two columns into one?

Answer: Use string concatenation or arithmetic:

```
 # Concatenate strings
df['Full_Name'] = df['First_Name'] + ' ' + df['Last_Name']

# Add numbers
df['Total'] = df['Price'] + df['Tax']

# Combine with separator
df['Address'] = df['City'] + ', ' + df['State']

# Using .str.cat() for more control
df['Full_Name'] = df['First_Name'].str.cat(df['Last_Name'], sep=' ')

# Handle NaN values
df['Full_Name'] = df['First_Name'].str.cat(df['Last_Name'], sep=' ', na_rep='')
```

Q11. How do you standardize column names?

Answer: Clean column names for consistency:

```
 # Lowercase all column names
df.columns = df.columns.str.lower()

# Replace spaces with underscores
df.columns = df.columns.str.replace(' ', '_')

# Remove special characters
df.columns = df.columns.str.replace('[^A-Za-z0-9_]', '', regex=True)

# Chain operations
df.columns = (df.columns
              .str.lower()
              .str.replace(' ', '_')
              .str.replace('[^a-z0-9_]', '', regex=True))
```

Q12. Why is data cleaning important before analysis?

Answer: Data cleaning is crucial because: Quality Issues:

- Garbage in, garbage out - bad data leads to bad insights
- Missing values can skew statistics
- Duplicates inflate counts and metrics
- Inconsistent formats cause errors

Analysis Impact:

- Incorrect conclusions from dirty data
- Models perform poorly on unclean data
- Visualizations misrepresent reality
- Business decisions based on flawed analysis

Common Problems:

- Duplicate records
- Missing values
- Inconsistent formatting
- Outliers and errors
- Wrong data types

Best Practice: Spend 60-80% of time on data cleaning before analysis.

5. GroupBy & Aggregation

Q1. What is `groupby()` in Pandas?

Answer:

`groupby()` groups rows with the same values in specified columns, allowing you to perform aggregate calculations on each group.

Concept:

- Split data into groups based on column values
- Apply function to each group independently
- Combine results into new DataFrame

Example:

```
# Group by single column
df.groupby('Department')['Salary'].mean()

# Group by multiple columns
df.groupby(['Department', 'City'])['Salary'].sum()
```

Learning Resources:

- Pandas GroupBy Documentation: https://pandas.pydata.org/docs/user_guide/groupby.html
- Real Python GroupBy Tutorial: https://realpython.com/pandas-groupby/

Q2. How do you calculate sum or mean using groupby?

Answer: Apply aggregate functions after groupby:

```
# Calculate mean
df.groupby('Department')['Salary'].mean()

# Calculate sum
df.groupby('Department')['Salary'].sum()

# Multiple columns
df.groupby('Department')[['Salary', 'Bonus']].mean()

# All numeric columns
df.groupby('Department').mean()
```

Q3. How do you count records per group?

Answer:

Use `count()` or `size()`:

```
# Count non-null values
df.groupby('Department')['Employee_ID'].count()

# Count all rows (including NaN)
df.groupby('Department').size()

# Count unique values
df.groupby('Department')['Employee_ID'].nunique()

# Value counts by group
df.groupby('Department')['Status'].value_counts()
```

Q4. How do you group data by multiple columns?

Answer: Pass a list of column names:

```
# Group by multiple columns
df.groupby(['Department', 'City'])['Salary'].mean()

# Multiple aggregations
df.groupby(['Department', 'City']).agg({
    'Salary': 'mean',
    'Age': 'median',
    'Employee_ID': 'count'
})

# Result has multi-level index
grouped = df.groupby(['Department', 'City']).mean()
```

Q5. Difference between `count()` and `size()`?

Answer:

`count()`:

- Counts non-null values
- Returns count per column
- Excludes NaN values

`size()` :

- Counts all rows in group
- Returns single number per group
- Includes rows with NaN

Examples:

```
# count() - excludes NaN
df.groupby('Department')['Salary'].count()

# size() - includes all rows
df.groupby('Department').size()

# If Salary has NaN values:
# count() returns lower number
# size() returns total rows
```

When to use:

- Use `count()` when you need non-null counts per column
- Use `size()` when you need total rows per group

Q6. How do you apply multiple aggregations at once?

Answer:

Use `agg()` method:

```
# Single aggregation
df.groupby('Department')['Salary'].agg('mean')

# Multiple aggregations on one column
df.groupby('Department')['Salary'].agg(['mean', 'median', 'std'])

# Different aggregations for different columns
df.groupby('Department').agg({
    'Salary': ['mean', 'max', 'min'],
    'Age': 'median',
    'Employee_ID': 'count'
})

# Custom function
df.groupby('Department')['Salary'].agg(lambda x: x.max() - x.min())
```

Q7. How do you rename aggregated columns?

Answer: Use tuple syntax in `agg()`:

```
# Rename columns
df.groupby('Department').agg(
    avg_salary=('Salary', 'mean'),
    total_employees=('Employee_ID', 'count'),
    max_age=('Age', 'max')
)

# Multiple aggregations with custom names
df.groupby('Department').agg(
    avg_salary=('Salary', 'mean'),
    min_salary=('Salary', 'min'),
    max_salary=('Salary', 'max')
)
```

Q8. How do you sort grouped results?

Answer:

Use `sort_values()` after grouping:

```
# Sort by aggregated value
result = df.groupby('Department')['Salary'].mean()
result.sort_values(ascending=False)

# Sort during aggregation
df.groupby('Department')['Salary'].mean().sort_values()

# Sort by group name
df.groupby('Department')['Salary'].mean().sort_index()

# Multiple sort keys
df.groupby('Department').agg({
    'Salary': 'mean',
    'Age': 'median'
}).sort_values(['Salary', 'Age'], ascending=[False, True])
```

Q9. How do you filter groups based on a condition?

Answer:

Use `filter()` method:

```
# Keep groups where condition is True
df.groupby('Department').filter(lambda x: x['Salary'].mean() > 60000)

# Keep departments with more than 5 employees
df.groupby('Department').filter(lambda x: len(x) > 5)

# Keep groups where max salary > 100000
df.groupby('Department').filter(lambda x: x['Salary'].max() > 100000)
```

Note: `filter()` returns the original rows that belong to qualifying groups.

Q10. How do you reset index after groupby?

Answer:

Use `reset_index()` :

```
# GroupBy creates index from grouped columns
result = df.groupby('Department')['Salary'].mean()

# Reset to regular columns
result.reset_index()

# Reset with custom name
result.reset_index(name='Average_Salary')

# Reset during aggregation
df.groupby('Department')['Salary'].mean().reset_index()

# Keep index
df.groupby('Department')['Salary'].mean()  # Department is index
```

Q11. How do you handle missing values in groupby?

Answer: Control NaN handling

with `dropna` parameter:

```
# Default: exclude NaN from grouping column
df.groupby('Department')['Salary'].mean()

# Include NaN as a separate group
df.groupby('Department', dropna=False)['Salary'].mean()

# NaN in aggregated column (automatically skipped)
df.groupby('City')['Salary'].mean()  # NaN salaries ignored

# Fill NaN before grouping
df['Department'].fillna('Unknown').groupby(df['Department']).mean()
```

Q12. When should you use groupby in real projects?

Answer: Use groupby when you

need: Business Analytics:

- Sales by region/product/month
- Customer segments analysis

- Department-wise metrics

Data Summarization:

- Average values per category
- Counts and totals by group
- Statistical summaries

Feature Engineering:

- Creating aggregated features for ML
- Encoding categorical variables
- Time-based aggregations

Common Use Cases:

```
# Sales analysis
sales.groupby('Region')['Revenue'].sum()

# Customer behavior
users.groupby('Segment')['Purchases'].mean()

# Time series
df.groupby(df['Date'].dt.month)['Sales'].sum()
```

---

6. Merging & Combining DataFrames

Q1. How do you merge two DataFrames in Pandas?

Answer:

Use `merge()` function:

```
# Basic merge (inner join on common columns)
pd.merge(df1, df2)

# Specify join column
pd.merge(df1, df2, on='Customer_ID')

#  Different  join  types  pd.merge(df1,
df2, on='ID', how='inner') pd.merge(df1,  # Default
df2,  on='ID',  how='left')  pd.merge(df1,
df2, on='ID', how='right') pd.merge(df1,
df2, on='ID', how='outer')
```

Learning Resources:

- Pandas Merge Documentation: https://pandas.pydata.org/docs/user_guide/merging.html
- Real Python Pandas Merge Tutorial: https://realpython.com/pandas-merge-join-and-concat/

Q2. What is the difference between `merge()` and `concat()`?

Answer:

`merge()`:

- Combines DataFrames based on common columns (like SQL joins)
- Matches rows using key columns
- Used for relational data

`concat()` :

- Stacks DataFrames vertically or horizontally
- Doesn't match on keys
- Used for combining similar structure data

Examples:

```
# merge() - join on key
pd.merge(orders, customers, on='customer_id')

# concat() - stack vertically
pd.concat([df1, df2], axis=0)  # Add rows

# concat() - stack horizontally
pd.concat([df1, df2], axis=1)  # Add columns
```

Q3. What is an inner join?

Answer: Inner join returns only rows where the key exists in both DataFrames.

Characteristics:

- Default join type
- Most restrictive
- Only keeps matching rows
- Can reduce row count

Example:

```
# Inner join
pd.merge(df1, df2, on='ID', how='inner')

# Only returns rows where ID exists in both df1 and df2
```

Visual:

```
df1: IDs [1, 2, 3]
df2: IDs [2, 3, 4]
Result: IDs [2, 3] # Only common IDs
```

Q4. What is a left join?

Answer: Left join returns all rows from left DataFrame and matching rows from right DataFrame.

Characteristics:

- Keeps all rows from left DataFrame
- NaN for non-matching right DataFrame columns
- Useful when left DataFrame is primary

Example:

```
# Left join
pd.merge(df1, df2, on='ID', how='left')

# All rows from df1, matched with df2 where possible
```

Visual:

```
df1: IDs [1, 2, 3]
df2: IDs [2, 3, 4]
Result: IDs [1, 2, 3]  # All from left, NaN for ID 1 from df2
```

Q5. What is a right join?

Answer: Right join returns all rows from right DataFrame and matching rows from left DataFrame.

Characteristics:

- Keeps all rows from right DataFrame NaN for non-
- matching left DataFrame columns Opposite of left join
-

Example:

```
# Right join
pd.merge(df1, df2, on='ID', how='right')

# All rows from df2, matched with df1 where possible
```

Visual:

```
df1: IDs [1, 2, 3]
df2: IDs [2, 3, 4]
Result: IDs [2, 3, 4]  # All from right, NaN for ID 4 from df1
```

Note: `df1.merge(df2, how='right')` is same as `df2.merge(df1, how='left')`

Q6. What is an outer join?

Answer: Outer join (full outer join) returns all rows from both DataFrames.

Characteristics:

- Most inclusive join type
- Keeps all rows from both DataFrames
- NaN where data doesn't match
- Combines left join + right join

Example:

```
# Outer join
pd.merge(df1, df2, on='ID', how='outer')

# All rows from both DataFrames
```

Visual:

```
df1: IDs [1, 2, 3]
df2: IDs [2, 3, 4]
Result: IDs [1, 2, 3, 4]  # All IDs from both, NaN where missing
```

Q7. What happens if merge keys have duplicates?

Answer: Duplicates in merge keys create Cartesian product for those keys:

Behavior:

- Each row in left matches ALL corresponding rows in right
- Can dramatically increase row count
- Creates one row for each combination

Example:

```
df1:
ID  Name
1 1 Alice
2   Bob
    Charlie

df2:
ID 1City
1 2 NYC
    LA
    Chicago

Result of merge:
ID  Name     City
1 1 Alice    NYC
1 1 Alice    LA
2   Bob  Bob NYC
    Charlie  LA
             Chicago
```

Rows multiplied: 2 × 2 = 4 rows for ID=1 Best Practice: Ensure merge keys are unique or understand multiplication effect.

Q8. How do you merge on multiple columns?

Answer: Pass a list of columns to `on` parameter:

```
# Merge on multiple columns
pd.merge(df1, df2, on=['Customer_ID', 'Date'])

# Rows match only if ALL specified columns match

# Example: Must match both customer AND date
orders.merge(returns, on=['Order_ID', 'Product_ID'])
```

Q9. How do you merge DataFrames with different column names?

Answer:

Use `left_on` and `right_on`:

```
# Different column names for join key
pd.merge(df1, df2,
         left_on='Customer_ID',
         right_on='Client_ID')

# Multiple columns with different names
pd.merge(df1, df2,
         left_on=['Cust_ID', 'Date'],
         right_on=['Client_ID', 'Order_Date'])

# Result includes both columns (Customer_ID and Client_ID)
# Drop duplicate column if needed
result.drop('Client_ID', axis=1)
```

Q10. How do you append rows to a DataFrame?

Answer:

Use `concat()` or `append()` (deprecated):

```
# Using concat (recommended)
pd.concat([df1, df2], ignore_index=True)

# Append single row (create DataFrame first)
new_row = pd.DataFrame({'Name': ['John'], 'Age': [30]})
df = pd.concat([df, new_row], ignore_index=True)

# Append multiple rows
new_rows = pd.DataFrame({
    'Name': ['John', 'Jane'],
    'Age': [30, 25]
})     df     =     pd.concat([df,     new_rows],
ignore_index=True)
```

Note: `df.append()` is deprecated. Use `pd.concat()` instead.

Q11. How do you combine DataFrames vertically?

Answer:

Use `concat()` with `axis=0` :

```
# Stack vertically (add rows)
pd.concat([df1, df2], axis=0)

# Reset index
pd.concat([df1, df2], axis=0, ignore_index=True)

# Only keep common columns
pd.concat([df1, df2], axis=0, join='inner')

# Keep all columns (fill NaN for missing)
pd.concat([df1, df2], axis=0, join='outer')  # Default
```

Use case: Combining data from multiple files/sources with same structure.

Q12. How do you combine DataFrames horizontally?

Answer:

Use `concat()` with `axis=1` :

```
# Stack horizontally (add columns)
pd.concat([df1, df2], axis=1)

#  Align  by  index  pd.concat([df1,  df2],
axis=1, join='inner') pd.concat([df1, df2], # Common indexes only
axis=1, join='outer')                       # All indexes

# Add suffix to duplicate column names
pd.concat([df1, df2], axis=1, keys=['left', 'right'])
```

Use case: Adding new features/columns from different sources.

---

7. Working with Dates & Time

Q1. How do you convert a column to datetime?

Answer:

Use `pd.to_datetime()` :

```
# Convert to datetime
df['Date'] = pd.to_datetime(df['Date'])

# Specify format
df['Date'] = pd.to_datetime(df['Date'], format='%Y-%m-%d')

# Handle errors
df['Date'] = pd.to_datetime(df['Date'], errors='coerce')  # Invalid → NaT

# Convert multiple columns
df['Start'] = pd.to_datetime(df['Start'])
df['End'] = pd.to_datetime(df['End'])
```

Learning Resources:

- Pandas Time Series Documentation: https://pandas.pydata.org/docs/user_guide/timeseries.html
- Real Python DateTime Tutorial: https://realpython.com/python-datetime/

Q2. How do you extract year, month, and day from dates?

Answer:

Use `.dt` accessor:

```
# Extract year
df['Year'] = df['Date'].dt.year

# Extract month
df['Month'] = df['Date'].dt.month

# Extract day
df['Day'] = df['Date'].dt.day

# Extract weekday (Monday=0, Sunday=6)
df['Weekday'] = df['Date'].dt.dayofweek

# Extract day name
df['Day_Name'] = df['Date'].dt.day_name()

# Extract month name
df['Month_Name'] = df['Date'].dt.month_name()

# Extract quarter
df['Quarter'] = df['Date'].dt.quarter
```

Q3. How do you sort data by date?

Answer:

Use `sort_values()` on date column:

```
# Sort by date (ascending)
df.sort_values('Date')

# Sort descending (most recent first)
df.sort_values('Date', ascending=False)

# Sort by multiple date columns
df.sort_values(['Start_Date', 'End_Date'])

# Sort and save
df.sort_values('Date', inplace=True)
```

Q4. How do you filter rows for a specific date range?

Answer: Use boolean indexing with date comparisons:

```
# Filter specific date
df[df['Date'] == '2024-01-15']

# Date range
df[(df['Date'] >= '2024-01-01') & (df['Date'] <= '2024-12-31')]

# Using between
df[df['Date'].between('2024-01-01', '2024-12-31')]

# Last 30 days
df[df['Date'] >= (pd.Timestamp.now() - pd.Timedelta(days=30))]

# Specific year
df[df['Date'].dt.year == 2024]

# Specific month
df[df['Date'].dt.month == 6]
```

Q5. How do you calculate difference between two dates?

Answer: Subtract datetime columns:

```
# Calculate difference
df['Duration'] = df['End_Date'] - df['Start_Date']

# Result is Timedelta type
# Extract days
df['Days'] = (df['End_Date'] - df['Start_Date']).dt.days

# Age calculation
df['Age'] = (pd.Timestamp.now() - df['Birth_Date']).dt.days // 365

# Difference in hours
df['Hours'] = (df['End'] - df['Start']).dt.total_seconds() / 3600
```

Q6. What is `Timedelta` in Pandas?

Answer:

`Timedelta` **represents a duration or difference between datetimes.**

Creation:

```
# Create Timedelta
pd.Timedelta(days=7)
pd.Timedelta(hours=2)
pd.Timedelta('2 days 3 hours')

# Add to datetime
df['Future_Date'] = df['Date'] + pd.Timedelta(days=7)

# Subtract from datetime
df['Past_Date'] = df['Date'] - pd.Timedelta(weeks=2)
```

Attributes:

```
td = df['End'] - df['Start']
td.days # Number of days
td.seconds # Seconds component
td.dt.total_seconds() # Total in seconds
```

Q7. How do you resample data by month or week?

Answer:

Use `resample()` with datetime index:

```
# Set datetime as index
df.set_index('Date', inplace=True)

# Resample by month
df.resample('M').sum()     # Monthly sum
df.resample('M').mean()    # Monthly average

# Resample by week
df.resample('W').sum()

# Resample by year
df.resample('Y').sum()

# Resample by day
df.resample('D').sum()

# Common frequency aliases:
# 'D': Day # 'W': Week #
'M': Month # 'Q': Quarter #
'Y': Year # 'H': Hour
```

Q8. How do you handle missing dates in time series?

Answer: Create complete date range and reindex:

```
# Create complete date range
date_range = pd.date_range(start='2024-01-01', end='2024-12-31', freq='D')

# Reindex to include all dates
df = df.set_index('Date').reindex(date_range)

# Fill missing values
df.fillna(0, inplace=True)  # Fill with 0
df.fillna(method='ffill', inplace=True)  # Forward fill
df.interpolate(inplace=True) # Interpolate


# Reset index
df.reset_index(inplace=True)
df.rename(columns={'index': 'Date'}, inplace=True)
```

Q9. How do you group data by month or year?

Answer:

Use `.dt` accessor with `groupby()`:

```
# Group by year
df.groupby(df['Date'].dt.year)['Sales'].sum()

# Group by month
df.groupby(df['Date'].dt.month)['Sales'].mean()

# Group by year and month
df.groupby([df['Date'].dt.year, df['Date'].dt.month])['Sales'].sum()

# Group by quarter
df.groupby(df['Date'].dt.quarter)['Revenue'].sum()

# Group by weekday
df.groupby(df['Date'].dt.dayofweek)['Sales'].mean()

# Create period column first
df['Year_Month'] = df['Date'].dt.to_period('M')
df.groupby('Year_Month')['Sales'].sum()
```

Q10. How do you find the earliest and latest dates?

Answer:

Use `min()` and `max()`:

```
# Earliest date
earliest = df['Date'].min()

# Latest date
latest = df['Date'].max()

# Date range
date_range = df['Date'].max() - df['Date'].min()
print(f"Data spans {date_range.days} days")

# Find row with earliest date
df[df['Date'] == df['Date'].min()]

# Find row with latest date
df.loc[df['Date'].idxmax()]
```

8. Basic Pandas Logic (Interview Thinking)

Q1. How do you create a new column using existing columns?

Answer: Direct assignment with operations:

```
 # Arithmetic operations
df['Total'] = df['Price'] * df['Quantity']
df['Profit'] = df['Revenue'] - df['Cost']

# String concatenation
df['Full_Name'] = df['First_Name'] + ' ' + df['Last_Name']

# Conditional logic
df['Category'] = df['Age'].apply(lambda x: 'Adult' if x >= 18 else 'Minor')

# Using numpy
df['Discount'] = np.where(df['Amount'] > 100, 0.1, 0.05)

# Multiple conditions
conditions = [
    df['Score'] >= 90,
    df['Score'] >= 80,
    df['Score'] >= 70
]   choices   =   ['A',   'B',   'C']   df['Grade']   =
np.select(conditions, choices, default='F')
```

Q2. How do you apply a function to a column?

Answer:

Use `apply()` method:

```
 # Apply function to column
df['Length'] = df['Name'].apply(len)

# Custom function
def categorize_age(age):
    if age < 18:
        return 'Minor'
    elif age < 65:
        return 'Adult'
    else:
        return 'Senior'

df['Age_Category'] = df['Age'].apply(categorize_age)

# Lambda function
df['Squared'] = df['Number'].apply(lambda x: x ** 2)

# Apply to multiple columns
df['Sum'] = df.apply(lambda row: row['A'] + row['B'], axis=1)
```

Q3. Difference between `apply()` and vectorized operations?

Answer: Vectorized Operations:

- Much faster (optimized C code) Work
- on entire arrays at once Preferred
- method Limited to simple operations
-

`apply()`:

- Slower (Python loops)
- Works row by row or column by column
- More flexible for complex logic
- Use when vectorization not possible

Examples:

```
 # Vectorized (FAST)
df['Total'] = df['Price'] * df['Quantity']  # Preferred

# apply() (SLOWER)
df['Total'] = df.apply(lambda row: row['Price'] * row['Quantity'], axis=1)

# Use vectorized when possible:
df['Doubled'] = df['Value'] * 2  # Fast
df['Doubled'] = df['Value'].apply(lambda x: x * 2)  # Slower
```

Rule: Always prefer vectorized operations. Use `apply()` only for complex custom logic.

Q4. How do you find unique values in a column?

Answer:

Use `unique()` or `nunique()`:

```
 # Get unique values (returns array)
df['City'].unique()

# Count unique values (returns number)
df['City'].nunique()

# Get unique values as list
df['City'].unique().tolist()

# Check if value exists
'NYC' in df['City'].values

# Unique values for multiple columns
df[['City', 'State']].nunique()
```

Q5. How do you count unique values?

Answer:

Use `nunique()` or `value_counts()`:

```
# Count unique values
df['Category'].nunique()

# Count occurrences of each unique value
df['Category'].value_counts()

# Include NaN in count
df['Category'].nunique(dropna=False)

# Percentage distribution
df['Category'].value_counts(normalize=True)

# Sort by value instead of count
df['Category'].value_counts().sort_index()
```

Q6. How do you check if a value exists in a column?

Answer:

Use `isin()` or boolean indexing:

```
# Check if single value exists
'NYC' in df['City'].values

# Check multiple values
df['City'].isin(['NYC', 'LA', 'Chicago']).any()

# Filter rows where value exists
df[df['City'] == 'NYC']

# Check if ANY row meets condition
(df['Age'] > 65).any()

# Check if ALL rows meet condition
(df['Age'] > 0).all()
```

Q7. How do you use `value_counts()`?

Answer:

`value_counts()` counts occurrences of unique values:

```
# Basic usage
df['Status'].value_counts()

# With percentages
df['Status'].value_counts(normalize=True)

# Include NaN
df['Status'].value_counts(dropna=False)

# Sort by index instead of count
df['Status'].value_counts().sort_index()

# Get top N values
df['Product'].value_counts().head(10)

# As DataFrame
df['Status'].value_counts().reset_index()
df['Status'].value_counts().to_frame()
```

Q8. How do you round numerical values?

Answer:

Use `round()`:

```
# Round to 2 decimal places
df['Price'].round(2)

# Round entire DataFrame
df.round(2)

# Round different columns differently
df.round({'Price': 2, 'Tax': 3, 'Total': 0})

# Round to nearest integer
df['Age'].round(0)

# Alternative: use numpy
np.round(df['Price'], 2)
```

Q9. How do you clip values within a range?

Answer:

Use `clip()` to limit values:

```
# Clip between min and max
df['Age'].clip(lower=0, upper=100)

# Set floor (minimum)
df['Score'].clip(lower=0)

# Set ceiling (maximum)
df['Percentage'].clip(upper=100)

# Clip entire DataFrame
df.clip(lower=0, upper=1000)

# Replace with clip
df['Age'] = df['Age'].clip(0, 120)
```

Use case: Handle outliers, ensure valid ranges.

Q10. How do you compare two columns?

Answer: Use comparison operators:

```
# Create boolean column
df['Is_Profit'] = df['Revenue'] > df['Cost']

# Compare equality
df['Match'] = df['Predicted'] == df['Actual']

# Difference
df['Difference'] = df['Value1'] - df['Value2']

# Percentage change
df['Pct_Change'] = ((df['New'] - df['Old']) / df['Old']) * 100

# Find rows where columns differ
df[df['Column1'] != df['Column2']]

# Count matching rows
(df['Col1'] == df['Col2']).sum()
```

Q11. How do you handle boolean conditions in Pandas?

Answer: Use boolean indexing and operators:

```
# Single condition
df[df['Age'] > 25]

# Multiple conditions with &, |, ~
df[(df['Age'] > 25) & (df['City'] == 'NYC')]  # AND
df[(df['Age'] < 18) | (df['Age'] > 65)]      # OR
df[~(df['Status'] == 'Active')]               # NOT

# Store condition
condition = (df['Salary'] > 50000) & (df['Experience'] > 3)
df[condition]

# Create boolean column
df['Is_Senior'] = df['Age'] > 60

# Count True values
df['Is_Senior'].sum()

# Using np.where
df['Category'] = np.where(df['Score'] > 80, 'Pass', 'Fail')

# Multiple conditions with np.select
conditions = [
    df['Score'] >= 90,
    df['Score'] >= 80,
    df['Score'] >= 70
]   choices   =   ['A',   'B',   'C']   df['Grade']   =
np.select(conditions, choices, default='F')
```

Free Learning Resources

Interactive Practice Platforms

1. Kaggle Learn Pandas Course - https://www.kaggle.com/learn/pandas
    - Interactive exercises
    - Real datasets

- Free certificates
- Beginner-friendly

2. DataCamp Pandas Tutorial - https://www.datacamp.com/community/tutorials/pandas-tutorial-dataframe-python

   - Hands-on tutorials
   - Step-by-step guidance
   - Practice problems

3. W3Schools Pandas Tutorial - https://www.w3schools.com/python/pandas/

   - Interactive examples
   - Try it yourself editor
   - Quick reference

4. Real Python Pandas Tutorials - https://realpython.com/learning-paths/pandas-data-science/

   - In-depth articles
   - Video tutorials
   - Practical examples

Comprehensive Documentation

5. Official Pandas Documentation - https://pandas.pydata.org/docs/

   - Complete API reference
   - User guide
   - Best practices

6. Pandas Getting Started - https://pandas.pydata.org/docs/getting_started/intro_tutorials/

   - Official tutorials
   - Step-by-step lessons
   - Practical examples

7. Pandas Cheat Sheet - https://pandas.pydata.org/Pandas_Cheat_Sheet.pdf

   - Quick reference
   - Common operations
   - Syntax guide

Video Tutorials

8. Corey Schafer Pandas Series - Search "Corey Schafer Pandas" on YouTube

   - Clear explanations
   - Beginner-friendly
   - Full series available

9. Keith Galli Pandas Tutorial - Search "Keith Galli Pandas" on YouTube

   - Complete walkthrough
   - Real-world examples
   - Project-based

10. freeCodeCamp Pandas Course - Search "freeCodeCamp Pandas" on YouTube

    - Full-length course
    - Comprehensive coverage
    - Free

Practice Datasets

11. Kaggle Datasets - https://www.kaggle.com/datasets

    - Thousands of datasets
    - Various domains
    - Practice challenges

12. UCI Machine Learning Repository - https://archive.ics.uci.edu/ml/index.php

    - Classic datasets
    - Well-documented
    - Research quality

13. Data.gov - https://data.gov/

    - Government datasets
    - Real-world data
    - Free to use

Jupyter Notebooks & Examples

19. Pandas Exercises - https://github.com/guipsamora/pandas_exercises

    - Practice problems with solutions
    - Organized by difficulty

    - GitHub repository

20. 100 Pandas Puzzles - https://github.com/ajcr/100-pandas-puzzles
    - Challenge problems
    - Solutions included
    - Test your skills

Interview Preparation Tips

Before the Interview

Understand concepts, don't just memorize

- Be able to explain WHY, not just HOW
- Understand trade-offs between different approaches
- Know when to use each method

Practice with real datasets

- Download datasets from Kaggle
- Perform actual data cleaning
- Create meaningful analyses

Review common patterns

- Data loading and inspection
- Filtering and selection
- Grouping and aggregation
- Merging datasets
- Handling missing values

Prepare examples from experience

- Have stories ready from past projects
- Explain your thought process
- Discuss challenges you've solved

Practice coding without IDE

- Many interviews use whiteboards or shared screens
- Focus on syntax accuracy
- Know common method names

During the Interview

Clarify requirements first

- Ask about data size and structure
- Understand expected output format
- Confirm edge cases to handle

Explain your approach

- Talk through your logic before coding
- Mention alternative approaches
- Explain trade-offs

Write clean, readable code

- Use meaningful variable names
- Add comments for complex logic
- Follow consistent style

Test your logic

- Walk through with sample data
- Check edge cases (empty DataFrames, NaN values)
- Verify output format

Discuss optimization

- Mention vectorization opportunities
- Explain memory considerations
- Suggest performance improvements

Common Interview Question Types

1. Data Loading & Inspection
    - Reading CSV/Excel files
    - Basic DataFrame operations
    - Checking data types and structure
2. Data Cleaning

- Handling missing values
- Removing duplicates Data
- type conversions String
- cleaning

3. Filtering & Selection

- Boolean indexing
- loc vs iloc
- Conditional filtering
- Column selection

4. Aggregation & GroupBy

- Basic aggregations
- Grouping by categories
- Multiple aggregations
- Custom functions

5. Merging & Joining

- Different join types
- Handling duplicate keys
- Combining multiple DataFrames

6. Data Transformation

- Creating new columns
- Applying functions
- Reshaping data

7. Performance Questions

- Vectorization vs apply()
- Memory optimization
- Efficient operations

8. Real-world Scenarios

- Business logic implementation
- Data pipeline design
- Error handling

Quick Reference Checklist

Before your interview, ensure you can explain:

Core Concepts

- ☐ DataFrame vs Series Creating DataFrames from different sources Basic
- ☐ DataFrame operations (head, tail, shape, info, describe) Data types in
- ☐ Pandas Index and columns
- ☐
- ☐

Data Selection

- ☐ Selecting columns (single and multiple) loc vs
- ☐ iloc Boolean indexing Filtering with conditions
- ☐ Sorting data
- ☐
- ☐

Missing Data

- ☐ Checking for missing values (isnull, isna)
- ☐ Counting missing values Dropping missing values
- ☐ (dropna) Filling missing values (fillna, ffill, bfill)
- ☐ When to drop vs fill
- ☐

Data Cleaning

- ☐ Removing duplicates Changing data
- ☐ types (astype) String operations (str
- ☐ accessor) Replacing values Splitting
- ☐ and merging columns
- ☐

GroupBy & Aggregation

- ☐ Basic groupby operations Common aggregations (sum,
- ☐ mean, count, etc.) count() vs size() Multiple aggregations
- ☐ with agg() Filtering groups Resetting index
- ☐
- ☐
- ☐

Merging & Combining

- ☐ merge() vs concat() Different join types (inner, left,
- ☐ right, outer) Merging on single and multiple
- ☐ columns Handling duplicate keys Combining
- ☐ vertically and horizontally
- ☐

Dates & Time

- ☐ Converting to datetime Extracting date components
- ☐ (year, month, day) Date filtering Date arithmetic
- ☐ Grouping by date periods
- ☐
- ☐

Applied Skills

- ☐ Creating new columns Applying functions
- ☐ (apply, lambda) Vectorized operations
- ☐ value_counts() Handling boolean
- ☐ conditions
- ☐

Performance

- ☐ Vectorization vs apply() Memory
- ☐ considerations Efficient data loading
- ☐ When to use different methods
- ☐

Your Next Steps

Immediate Actions (Today)

1. Bookmark this guide for quick reference 2. Install Pandas: pip install pandas
3. Try 5 basic commands in a Jupyter notebook
4. Download a small dataset from Kaggle

This Week

1. Complete Kaggle Learn Pandas course (4-5 hours) 2. Practice with 2-3 datasets 3. Implement 10 operations from this guide 4. Join Pandas community on Stack Overflow

Before Interview

1. Review all sections of this guide 2. Practice 20+ interview problems 3. Time yourself solving problems
4. Review your past projects using Pandas 5. Prepare 2-3 examples to discuss

After Interview

1. Note questions you struggled with 2. Practice weak areas

3. Learn from mistakes 4. Keep practicing regularly

Final Checklist

Technical Preparation:

- [ ] Can create DataFrames from various sources
- [ ] Comfortable with data selection (loc, iloc, boolean indexing)
- [ ] Know how to handle missing values
- [ ] Understand groupby operations
- [ ] Can merge DataFrames
- [ ] Familiar with date/time operations
- [ ] Know difference between apply() and vectorization
- [ ] Can write clean, efficient code

Interview Skills:

- [ ] Can explain concepts clearly
- [ ] Ask clarifying questions
- [ ] Test solutions with edge cases
- [ ] Discuss trade-offs
- [ ] Handle mistakes gracefully
- [ ] Show problem-solving process

Practical Experience:

- [ ] Completed practice problems
- [ ] Worked with real datasets
- [ ] Built small projects
- [ ] Debugged common errors
- [ ] Optimized code for performance

Key Takeaways

1. Master the basics first - DataFrame creation, selection, filtering 2. Practice with real data - Kaggle datasets, real problems 3. Think about performance - Vectorization over apply() 4. Handle edge cases - Missing values, empty DataFrames 5. Explain your approach - Interviews test thinking, not just coding

What Sets You Apart

- Practical experience with real datasets Understanding
- of performance implications Ability to explain complex
- concepts simply Problem-solving approach to new
- challenges Clean, readable code that others can
- maintain
